Risk Alert Classifier

Theory Concepts & Final Report

Early-warning system to identify high-risk customers of a digital banking platform. Companion document to the Jupyter Notebook *Risk_Alert_Classifier.ipynb*.

1. Theory Concepts and Definitions (Part A)

1. Logistic Regression

A classification algorithm that takes a weighted sum of the input features and passes it through the **sigmoid function** $p = 1 / (1 + e^{-z})$, giving a probability between 0 and 1 that a customer is high-risk. If $p \geq 0.5$ the customer is predicted as class 1. It suits classification because it outputs probabilities, trains fast, and its coefficients are easy to explain.

2. Classification performance metrics

Metrics such as the confusion matrix, accuracy, precision, recall, F1-score and AUC-ROC measure how well predicted classes match real classes. **Accuracy alone is insufficient** on imbalanced data: when ~12% of customers are high-risk, a model that predicts 'Low Risk' for everybody is ~88% accurate yet catches no risky customer.

3. Type-I and Type-II error

Type-I error (False Positive): a low-risk customer is flagged as high-risk, causing unnecessary reviews and customer annoyance. **Type-II error (False Negative):** a high-risk customer is predicted as low-risk, causing a missed default or fraud and direct financial loss. For a bank, Type-II is normally the costlier error.

4. Precision, Recall, F1, TPR and FPR

Precision = TP / (TP + FP): of the customers flagged, how many were truly high-risk. **Recall (= TPR)** = TP / (TP + FN): of the truly high-risk customers, how many were caught. **F1-Score** = 2 × Precision × Recall / (Precision + Recall): the harmonic mean of the two. **FPR** = FP / (FP + TN): share of low-risk customers wrongly flagged.

5. AUC-ROC

The ROC curve plots TPR against FPR for every possible decision threshold. The **AUC** is the area under this curve: 0.5 means random guessing, 1.0 means perfect ranking. It is threshold-independent and shows how well a model ranks risky customers above safe ones, which makes it ideal for comparing classifiers.

6. Why imbalanced data is a problem

With very few minority examples the model minimises overall error by favouring the majority class. The result is high accuracy but low recall for the high-risk class. Remedies: resampling (under-sampling, over-sampling, SMOTE, ADASYN), class weights, and using recall / F1 / AUC instead of accuracy.

Resampling techniques used

- **Under-Sampling:** randomly removes majority-class rows.
- **Over-Sampling:** randomly duplicates minority-class rows.
- **SMOTE:** creates synthetic minority rows by interpolating between a minority point and its nearest neighbours.

- **ADASYN:** like SMOTE, but generates more synthetic rows in regions where the minority class is harder to learn.
- Resampling is applied **only to the training set**; the test set is never changed.

2. Dataset and Method

- **Data:** 4,600 customers, 17 raw features (demographic, transaction, credit behaviour). Target *risk_status*: 0 = Low Risk (4,043), 1 = High Risk (557, about 12.1%).
- **Features:** dropped *customer_id*; converted *last_transaction_date* into *days_since_last_txn*.
- **Split:** 80/20 stratified train-test split (3,680 / 920); high-risk share is the same in both.
- **Missing values:** about 2-5% missing in 7 columns. Numeric columns: StandardScaler + **KNN Imputer (k=5)**; categorical columns: most-frequent fill + one-hot encoding. All fitted on training data only (no leakage).
- **Models:** Logistic Regression, Decision Tree, Random Forest; tuned with Randomized Search CV and Grid Search CV (F1 scoring, stratified CV).
- **Random seed:** 42 everywhere for reproducibility.

Important data observation

Logistic Regression already scores almost perfectly on this dataset (5-fold CV AUC-ROC = 0.9995). We checked for duplicates and target leakage: there are none, and no single feature explains the target alone (best single-feature AUC = 0.953, *avg_late_payment_days*). The label is therefore almost a clean linear combination of several features, which is typical of a synthetic practice dataset. Real banking data would give lower scores. Because of this, the effect of class imbalance is only visible on a harder problem, so a **stress test** with 4 weaker features was added (Section 3.2).

3. Results

3.1 Baseline and balancing techniques (all features)

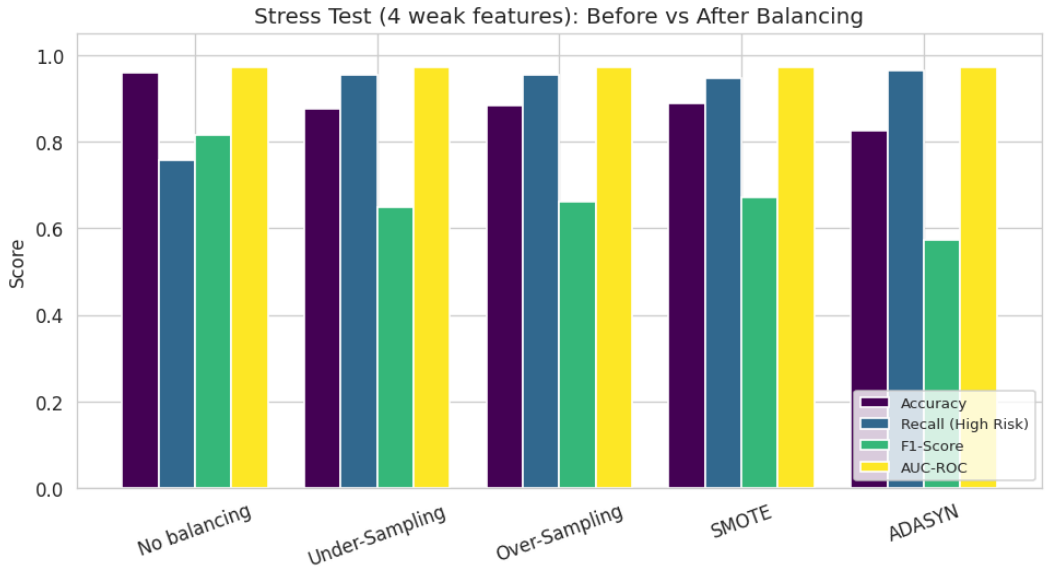
Model	Precision	Recall	F1-Score	AUC-ROC	FP (T-I)	FN (T-II)
Logistic Regression (baseline)	1.000	1.000	1.000	1.000	0	0
LogReg + Under-Sampling	0.982	1.000	0.991	1.000	2	0
LogReg + Over-Sampling	0.982	1.000	0.991	1.000	2	0
LogReg + SMOTE	0.982	1.000	0.991	1.000	2	0
LogReg + ADASYN	0.982	1.000	0.991	1.000	2	0

With all features the baseline is already perfect on the test set (recall = 1.0), so balancing cannot improve recall; it only adds a couple of false positives and lowers precision slightly. AUC-ROC is unchanged.

3.2 Stress test: imbalance on a harder problem (4 features)

Features kept: annual_income_inr, debt_balance_inr, complaints_last_6m, age. Model: Logistic Regression.

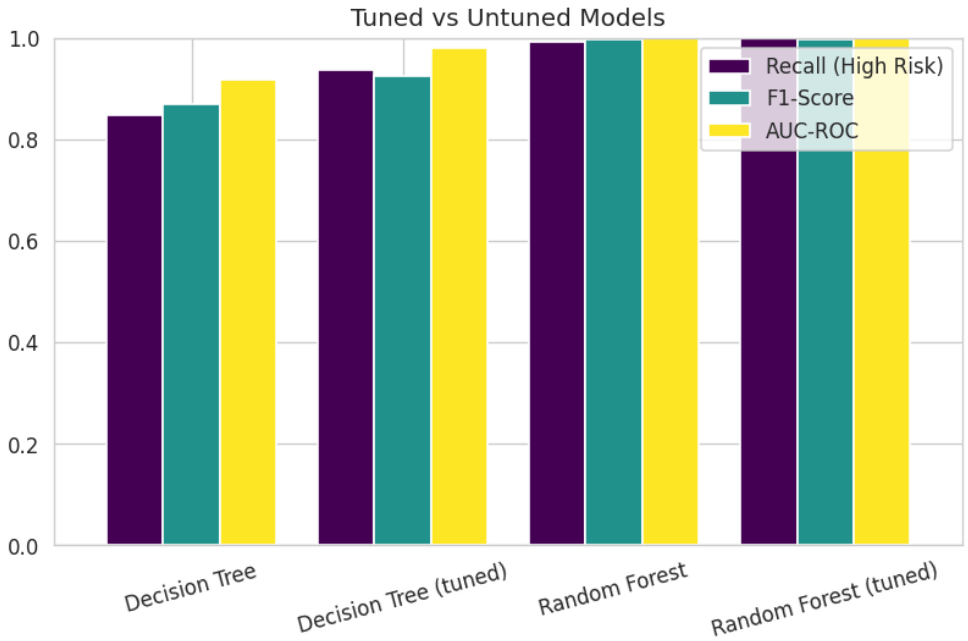
Technique	Accuracy	Precision	Recall	F1-Score	AUC-ROC
No balancing	0.959	0.884	0.757	0.816	0.973
Under-Sampling	0.875	0.491	0.955	0.648	0.972
Over-Sampling	0.883	0.507	0.955	0.662	0.973
SMOTE	0.888	0.520	0.946	0.671	0.973
ADASYN	0.826	0.407	0.964	0.572	0.973



This is the classic imbalance pattern: without balancing, accuracy is high but recall is low (risky customers are missed). All four balancing techniques raise recall sharply, at the cost of precision and accuracy (more Type-I errors). AUC-ROC stays the same because balancing moves the decision threshold rather than improving ranking. ADASYN gives the highest recall but the lowest precision.

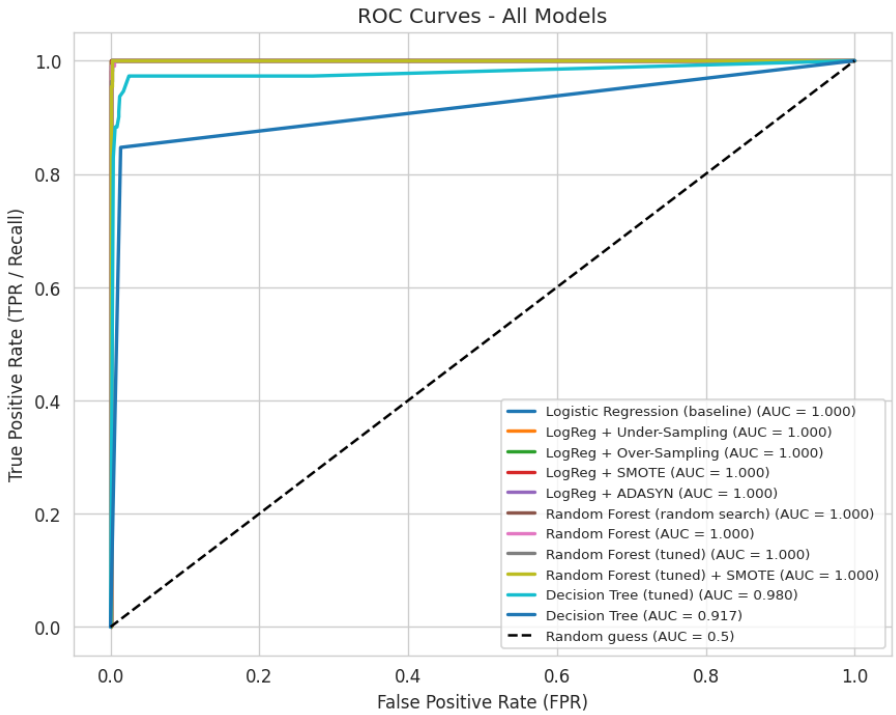
3.3 Decision Tree vs Random Forest, before and after tuning

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC	FN (T-II)
Decision Tree	0.970	0.895	0.847	0.870	0.917	17
Decision Tree (tuned)	0.982	0.912	0.937	0.924	0.980	7
Random Forest	0.999	1.000	0.991	0.996	1.000	1
Random Forest (tuned)	0.999	0.991	1.000	0.996	1.000	0



- The unrestricted **Decision Tree overfits**: 100% training accuracy but only 97% test accuracy and 84.7% test recall.
- The **Random Forest generalises far better** (train-test accuracy gap about 0.1 percentage points, test AUC about 1.0).
- **Tuning** (Randomized Search CV then Grid Search CV) helps the Decision Tree a lot (recall 0.85 to 0.94, AUC 0.92 to 0.98). The Random Forest was already near-perfect, so tuning changes little: differences of 0 or 1 customer are noise.

3.4 All models, ROC and AUC



Model	Precision	Recall	F1-Score	AUC-ROC	FP (T-I)	FN (T-II)
Logistic Regression (baseline)	1.000	1.000	1.000	1.000	0	0
LogReg + Under-Sampling	0.982	1.000	0.991	1.000	2	0

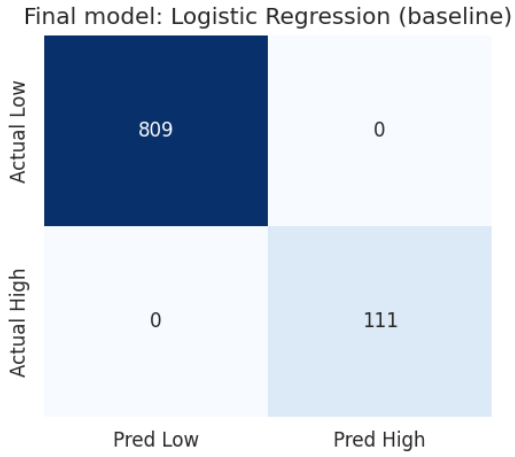
Model	Precision	Recall	F1-Score	AUC-ROC	FP (T-I)	FN (T-II)
LogReg + Over-Sampling	0.982	1.000	0.991	1.000	2	0
LogReg + SMOTE	0.982	1.000	0.991	1.000	2	0
LogReg + ADASYN	0.982	1.000	0.991	1.000	2	0
Random Forest (random search)	0.991	0.991	0.991	1.000	1	1
Random Forest	1.000	0.991	0.996	1.000	0	1
Random Forest (tuned)	0.991	1.000	0.996	1.000	1	0
Random Forest (tuned) + SMOTE	0.991	0.991	0.991	1.000	1	1
Decision Tree (tuned)	0.912	0.937	0.924	0.980	10	7
Decision Tree	0.895	0.847	0.870	0.917	11	17

4. Best Model Selection

Business requirement: **minimise false negatives** (missed high-risk customers). Rule: keep models whose AUC-ROC is within 0.03 of the best, pick the highest recall, break ties by F1, then prefer the simpler model.

Selected model: Logistic Regression (baseline) (test set: 111 of 111 high-risk customers caught, 0 missed, 0 false alarms).

The tuned Random Forest is practically tied (recall 1.000, 1 false positive) and is an equally valid choice, especially if feature importances are wanted. A single Decision Tree is the weakest option because of overfitting.



5. Business Interpretation and Recommendations

False Positive (Type-I): a safe customer is flagged. Cost is a manual review or a call. Cheap per case, but too many waste analyst time and hurt customer experience.

False Negative (Type-II): a risky customer is missed. Cost is unpaid loans, chargebacks or fraud losses. Much more expensive, so the bank should accept some extra false positives to catch more risky customers.

Recommendations

- Evaluate with **Recall, F1 and AUC-ROC**, never accuracy alone.
- The test set contains only 111 high-risk customers, so a 1-2 customer difference between models is **not statistically meaningful**.

- Near-perfect scores are unusual for real data. **Validate on fresh real data** before production; if scores are lower, use SMOTE or class weights as shown in the stress test.
- Send flagged customers to a **review queue** instead of automatic rejection.
- **Monitor** monthly and retrain when customer behaviour changes. Next steps: tune the probability threshold; try Gradient Boosting / XGBoost; add more behavioural features.

Note: numbers in this report come from one run with random seed 42. Re-running the notebook with the real imbalanced-learn library may change resampling results by small amounts.